DietView technical reference

Package companion — what DietView contains and why

Source: `vignettes/articles/technical-reference.Rmd`
`technical-reference.Rmd`

This is the **technical companion** to DietView. It is distinct from the journal article — which argues *why* the package matters — and from the auto-generated function reference. It explains, in one place, what the package really contains: every exported function, the logic of the code, the data structures it passes around, and the design decisions behind them.

What DietView is

DietView turns a **plain long table of diet records** — one row per prey record in a sampling unit (a stomach, a scat, a pellet, a DNA sample), with columns for the predator, the unit, the prey, an optional quantity, and any covariates — into interactive, taxonomically complete **sunburst** views of predator diet. The input table carries *no taxonomy*: DietView adds the seven flagship Linnaean ranks (kingdom to species) itself, from WoRMS (marine prey), the GBIF Backbone (all kingdoms), lineage strings already in the data, or a lineage table you supply. From that single input it produces per-taxon importance in two currencies, an explicit "NA" category for unidentified prey, covariate stratified views, an identification-resolution diagnostic (by any grouping, and by digestion stage), and three outputs (a Shiny explorer, a self-contained HTML dashboard, and plain plotly figures). The quantity can be prey mass, sequence reads, or relative read abundance (RRA), and every label follows.

Design principles (the why behind the code)

One input contract, spec-driven. DietView never hard-codes column names. You describe the roles of your columns once, with `dv_spec()`, and every other function reads that spec. This is what lets the same code serve any survey.

Taxonomy on demand, provenance-logged, never silently guessed. Prey are resolved to WoRMS or to the GBIF Backbone in a strict order — known ID, exact name, then optionally fuzzy name (GBIF adds a higher-rank match), then unresolved — and every prey carries a provenance record. Fuzzy, higher-rank, low-confidence and homonym matches are flagged, and `dv_taxonomy_report()` surfaces exactly what needs a human eye. Unaccepted names are followed to their valid taxon; results are cached, one file per authority. A GBIF name shared by several taxa (Polychaeta is a class of worms and a genus of flies) is resolved only when exactly one exact, accepted candidate exists; otherwise GBIF's conservative answer (the shared ancestor) is kept and flagged.

The explicit-NA philosophy. Where identification stops, DietView keeps the deeper ranks as `NA` and renders them as an explicit "NA" category (white). Two consequences: within-rank percentages still sum to 100 %, and identification gaps stay visible instead of being absorbed into a functional grouping.

Within-rank importance as a genuine partition. Importance is the share of each node within its own rank, so every rank sums to 100 % independently — the NA node included. Each ring of the sunburst is its own complete pie.

Two currencies. Every view can be expressed by occurrence (how often) or weight (how much biomass); they routinely disagree, and showing both is part of an honest diet description. A `count` mode sizes segments by distinct diet items.

Say what the quantity is. Sequence reads are not biomass. `dv_spec(weight_type =)` declares the weight column as prey mass (`"mass"`, %W), pooled reads (`"reads"`, share of all reads) or relative read abundance (`"rra"`): the reads of each unit are divided by the unit total before summing, so every unit weighs the same and the share at a rank is the mean RRA over units (Deagle et al. 2019). The arithmetic of the tree is unchanged; only the labels of the weight views differ, so no figure can call reads "weight".

Say what the unit is. `dv_spec(unit_label =)` names the sampling unit ("scat", "sample", "haul"); messages, printouts, the dashboard and the Shiny app use it. Argument and column names (`stomachs =`, `min_stomachs`, `n_stomachs`) stay the same, so code written for stomachs runs unchanged on scats.

The roll-up invariant. Aggregation is done at the finest rank and rolled up, so a parent's value equals the sum of its children; a unit test guards it. The sunburst passes only the leaf values to plotly (`branchvalues = "remainder"`, parents at 0), so the wedge sizes cannot drift from the table.

Fixed orientation for comparability. Sunbursts use `sort = FALSE` and a fixed start angle, so a taxon sits in the same place across covariate cells and cells can be compared directly.

Self-contained, reproducible outputs. The HTML dashboard embeds images as base64 and can render a single self-contained file; taxonomy can be frozen into a lineage table for a fully offline run.

Architecture and data flow

```
raw long table  +  dv_spec()                 -> dietview_spec
      |
      v
(wide table -> dv_from_wide();  lineage string -> dv_parse_lineage())
      |
      v
dv_prepare(data, spec, taxonomy=)            -> dietview_prep
   |   taxonomy: 'columns' | 'table' | 'worms' | 'gbif' | 'auto'
   |   weight_type = 'rra': reads -> per-unit proportions
      |
      v
dv_build_tree(prep$data, ranks, weight/count) -> dietview_tree  (within-rank %)
      |
      +-------------------+--------------------+
      v                   v                    v
dv_sunburst()      dv_rank_table()   dv_resolution() / dv_digestion_diagnostic()
dv_build_html()/dv_deploy()/run_dietview()
```

The stages: (0) reshape other data shapes — `dv_from_wide`, `dv_parse_lineage`; (1) declare & validate — `dv_spec`, `dv_validate`; (2) attach taxonomy — `dv_prepare`, `dv_taxonomy_worms`, `dv_taxonomy_gbif`, `dv_taxonomy_report`, `dv_lineage_check`; (3) build covariates — `dv_add_period`, `dv_size_bins`; (4) compute metrics — `dv_build_tree`, `dv_tree`, `dv_rank_table`; (5) draw figures — `dv_sunburst`, `dv_palette`; (6) identification resolution — `dv_resolution`, `dv_digestion_diagnostic`; (7) species cards — `dv_species_card_assets`; (8) ship — `dv_build_html`, `dv_deploy`, `run_dietview`, `dv_script`; (9) explore and export — `dv_summary`, `dv_prey_accumulation`, `dv_fo_table`, `dv_aggregate_threshold`, `dv_matrix`.

Data structures

`dietview_spec` (from `dv_spec()`): the role → column mapping plus display ranks — `predator`, `stomach`, `prey_name`/`prey_id`, `weight`, `count`, `digestion`, `covariates`, `ranks`, `predator_latin`, `weight_type` (`"mass"`, `"reads"`, `"rra"`) and `unit_label`. After `dv_prepare()` with `weight_type = "rra"`, `weight` names the new proportion column and `weight_raw` the original reads.

`dietview_prep` (from `dv_prepare()`): `data` (long table with rank columns attached), `spec`, `ranks`, `taxonomy_source`, `taxonomy_log` (NULL unless taxonomy = `"worms"` or `"gbif"`; its ID column is `aphia_used` or `gbif_key`, its attribute `source` names the authority) and, optionally, `stomachs` (the examined units). Its `print()` summarises counts (in the spec's unit), ranks, covariates, and the match breakdown.

`dietview_tree` (from `dv_build_tree()`): one row per sunburst node — `id`, `label`, `parent`, `rank`, `value_item`, `value_occ`, `value_w`, `pct_occ`, `pct_w` (each rank's percentages sum to 100), with the attributes `currency` and `weight_type`.

Function reference (summary)

The complete per-argument reference lives on the reference index. In brief:

- **`dv_from_wide()`** — pivot a wide sample-by-taxon table (taxa in rows or in columns) to the long format, drop zeros, join sample metadata, and optionally sum the rows of one taxon within a unit (`collapse_by`).
- **`dv_parse_lineage()`** — split pipeline lineage strings into rank columns: prefixed (`k__`, `p__`…) or positional fields, placeholders and `sp.` to `NA`, NCBI tax IDs and underscores cleaned, `s__epithet` completed with its genus; adds the deepest name as `prey_name`.
- **`dv_spec()`** — declare column roles once, with the quantity type and the unit label; a defensive constructor that errors if no prey key is given, or if reads are declared without a weight column.
- **`dv_validate()`** — check a table against a spec; hard errors on missing required columns, soft warnings on missing weight/digestion.
- **`dv_prepare()`** — validate + attach lineage via `columns` / `table` / `worms` / `gbif` / `auto`; normalise "NA"/"" to real `NA`; label lineage gaps; turn reads into RRA when asked.
- **`dv_taxonomy_worms()`** — resolve prey to WoRMS (id/exact/fuzzy/unresolved), follow unaccepted names to valid AphiaID, cache, log provenance.
- **`dv_taxonomy_gbif()`** — the same for the GBIF Backbone (all kingdoms), over the public GBIF API with no extra package: key, exact, fuzzy, higher-rank, homonyms settled from GBIF's alternatives when one exact accepted candidate exists; a step up to a kingdom alone counts as no match.
- **`dv_taxonomy_report()`** — return the matches to review (fuzzy, higher rank, low confidence, homonyms, unresolved, synonyms), sorted by records affected; optional CSV.
- **`dv_add_period()`** / **`dv_size_bins()`** — derive period and length-bin covariates with auto labels.
- **`dv_build_tree()`** — the metrics core: roll leaves up rank by rank, guarantee the total invariant, compute within-rank shares in both currencies.
- **`dv_rank_table()`** — the sorted importance table for one rank, NA included.
- **`dv_sunburst()`** — render the multi-rank sunburst; fixed orientation, currency-dependent sizing and hover.
- **`dv_palette()`** — stable label → colour dictionary; NA forced white.
- **`dv_resolution()`** — finest-resolved-rank shares of records and weight (via a `max.col` trick), overall or by any grouping columns; needs no digestion column.
- **`dv_digestion_diagnostic()`** — the same by digestion level, optionally crossed with a covariate.
- **`dv_species_card_assets()`** — resolve an ID photo + description as a base64 data URI; graceful offline fallback.
- **`dv_build_html()`** / **`dv_deploy()`** — assemble and render the self-contained dashboard (searchable cards + per-predator tabs, up to three bands).
- **`run_dietview()`** — the interactive Shiny explorer.
- **`print.dietview_prep()`** — compact console summary.

The metrics core, in detail

`dv_build_tree()` carries three quantities through the tree — `value_item` (distinct items), `value_occ` (occurrence), `value_w` (weight). Missing ranks are filled with the explicit "NA" string so they become real nodes. Records are summarised at the finest rank into leaves, then the function loops from the coarsest rank inward, grouping leaves by the prefix of ranks and summing:

```
lvl <- lapply(seq_along(ranks), function(i) {
  cols_i <- ranks[seq_len(i)]
  g <- summarise(group_by(leaves, across(all_of(cols_i))),
                 value_item = sum(value_item),
                 value_occ  = sum(occ),
                 value_w    = sum(w))
  g$id     <- do.call(paste, c(g[cols_i], sep = '|'))
  g$parent <- if (i == 1) '' else do.call(paste, c(g[cols_i[-i]], sep = '|'))
  g$rank   <- rank_labels[i]
  g
})
```

Because each level is a roll-up of the same leaves, a parent's value equals the sum of its children (the total invariant). Within each rank, the occurrence and weight totals become the denominator, so `pct_occ` and `pct_w` give each node's share within its rank.

The sunburst call

```
plot_ly(tree, ids = ~id, labels = ~label, parents = ~parent, values = size_vals,
        type = 'sunburst', branchvalues = 'total', sort = FALSE, rotation = rotation,
        textinfo = 'text', text = ~text,   # blank on unidentified ("NA") wedges
        hovertext = ~hovertext, hovertemplate = '%{hovertext}<extra></extra>',
        marker = list(colors = ~color))
```

`sort = FALSE` plus a fixed `rotation` keep the reading order and start angle identical across covariate cells — what makes side-by-side comparison meaningful.

The resolution trick

For each record, the finest resolved rank is found by multiplying a logical matrix of "rank is a real taxon" (non-NA and not a lineage-gap label) by the rank position and taking `max.col(..., ties = 'last')`; records with no resolved rank become "Unresolved". Grouping by any columns (`dv_resolution()`) or by digestion level (`dv_digestion_diagnostic()`) then gives record and weight shares within each group. For DNA data this measures where the marker or the reference library stops separating taxa; for stomachs, where digestion does.

Relative read abundance

With `weight_type = "rra"`, `dv_prepare()` divides each record's reads by the total of its unit (`stats::ave(reads, unit, FUN = sum)`) and stores the result as a new column that becomes the spec's weight. Summed in the tree, a node's `value_w` is the sum of per-unit proportions and `pct_w` is that sum over the number of units: the mean RRA. Units with no reads contribute nothing, with a warning. The labels of the weight views (`.dv_wlabels()`) follow the type.

Internal helpers and constants

`.DV_RANKS` (default kingdom..species), `.DV_D3` (the d3 category-20 palette), `.dv_titlecase()` / `.dv_slugify()` (display and file-name formatting), `.dv_fill_na()` (the explicit-NA conversion), `.dv_empty_tree()` (typed empty tree), `.dv_pct_fmt()` (hover formatting), `.dv_wlabels()` (labels of the weight views per `weight_type`), `.dv_unit()` (the unit label, singular or plural), `.dv_resolution_table()` (the resolution core), `.dv_add_rra()` (reads to RRA), `.dv_gbif_get()` / `.dv_gbif_try()` (GBIF requests with retries; mocked in tests), and the dashboard pieces `.dv_head_tags()`, `.dv_home()`, `.dv_render()`.

Bundled data and the input contract

One row = one prey record in one sampling unit; free column names mapped via `dv_spec()`. The package ships four kinds of data.

The ICES datasets (`data/`, lazy-loaded; documented in `?ices_diet`), the data of the articles, vignettes and examples. They are a public extract of the ICES Stomach Content Database (ICES, Copenhagen), distributed with the R package stomachr (Lindmark 2026), and built by `data-raw/ices_data.R`.

Dataset	Rows	Content
`ices_diet`	9,639 prey records	4,231 stomachs of 22 predators, North Sea, Skagerrak-Kattegat and Baltic Sea, 2020-2024; prey

Dataset	Rows	Content
		AphiaID and name, `item_type` (prey / unidentified / non_food), digestion code and label, count, weight, length, haul, date, position, region, predator length and weight
`ices_stomachs`	8,886 stomachs	every sampled stomach of 23 predators, 1,197 flagged empty: the sampling frame for the share of empty stomachs
`ices_lineage`	461 AphiaIDs	the frozen WoRMS lineage (kingdom to species, plus `scientific_name` and `worms_rank`), keyed by `prey_aphia_id` for `taxonomy = "table"`

The data are kept close to the ICES export: prey identified to very uneven depth, 1,553 records of unidentified material, 67 non-food items and about 1,000 nematode records (mostly parasites) kept for the analyst to decide on, and digestion stage missing for many records. The digestion labels are inferred from the older ICES format. The standard set-up used throughout the documentation drops non-food items and nematodes:

```
phylum <- ices_lineage$phylum[match(ices_diet$prey_aphia_id, ices_lineage$prey_aphia_id)]
dat <- ices_diet[ices_diet$item_type != "non_food" & !phylum %in% "Nematoda", ]
spec <- dv_spec(predator = "predator_common", predator_latin = "predator_latin",
                stomach = "stomach_id", prey_id = "prey_aphia_id",
                prey_name = "prey_name", weight = "prey_weight_g",
                count = "prey_count", digestion = "digestion_label",
                covariates = c("region", "year", "quarter"))
prep <- dv_prepare(dat, spec, taxonomy = "table", lineage = ices_lineage,
                   verbose = FALSE)
```

The Salish Sea harbour seal DNA data (`data/`, documented in `?salish_seal_dna`), built by `data-raw/salish_seal.R` from the public dataset of Thomas et al. (2022, *Scientific Data* 9: 68; figshare, CC0).

Dataset	Rows	Content
`salish_seal_dna`	11,641 prey records	4,625 scats from 52 haul-outs, 2012-2018; prey label, common name, 16S and COI reads, the authors' diet fraction (RRA), site, position, year, month, season, laboratory
`salish_seal_lineage`	117 labels	the frozen WoRMS lineage, keyed by `prey_label`; labels listing alternatives ("A OR B") keep only the ranks their candidates share

It is the worked example of the "Beyond stomachs" article: scats as units (`unit_label = "scat"`), `diet_fraction` as RRA, and `dv_resolution()` by laboratory.

The small example (`inst/extdata/dietview_example.csv`, 24 invented records, with `dietview_example_lineage.csv` and a data dictionary) stays for the unit tests and for checking the mechanics on a table you can read in full. It includes class-only, order/family-only and fully unidentified prey.

The simulated survey (`inst/extdata/dietview_sim.csv` and its lineage), built by `data-raw/simulate_known_diet.R`, has a known true diet; the "A known diet" vignette uses it to show what keeping or dropping unidentified prey does, which real data cannot show.

Dependencies

`dplyr` (aggregation), `plotly` (sunburst), `htmltools` (dashboard HTML), `jsonlite` (Wikipedia/Commons and the GBIF API), `knitr` (`image_uri`, vignettes), `grDevices` / `stats` / `tools` / `utils` (colours, stats, paths, web requests). Suggested: `worrms` (WoRMS), `shiny` + `DT` (app), `rmarkdown` (self-contained render), `testthat` + `withr` (tests). GBIF needs no extra package.

Testing

The testthat suite locks the invariants that make the sunburst coherent: within-rank shares sum to 100 (NA included); the explicit “NA” node exists; the roll-up invariant holds; empty input yields a typed empty tree, not an error. WoRMS and GBIF are tested offline, against mocked answers that copy the shape of each API (synonyms, fuzzy and higher-rank matches, homonyms, failures that must not be cached). The adapters, RRA arithmetic (against hand-computed means), `dv_resolution()` and the unit label are tested on small tables.

Extending DietView

Any survey works once it is shaped to one-row-per-prey-record and mapped with `dv_spec()`. Freeze taxonomy into a lineage table for a fully offline run; pass custom `ranks` or `colors`; add covariates and derive periods/size bins with the helpers; use GBIF for non-marine prey. See the Adapting DietView article, and Beyond stomachs for DNA metabarcoding, scats and other compositional data. Stable isotopes and fatty acids, which estimate source contributions rather than identify prey, are out of scope.

References

Deagle, B. E. et al. (2019). Counting with DNA in metabarcoding studies: How should we convert sequence reads to dietary data? *Molecular Ecology*, 28(2), 391-406.

Thomas, A. C. et al. (2022). Data on the diets of Salish Sea harbour seals from DNA metabarcoding. *Scientific Data*, 9, 68.

On this page

Developed by Sosthène A. V. Akia, CADI Gulf Center.

Site built with pkgdown 2.2.1.